

CHAPTER 6 — CHECKING A PERFECT NUMBER

SLIDE 1 — WHAT IS A PERFECT NUMBER?

MEET PERFECT NUMBERS!

A **Perfect Number** is a number whose **proper factors add up to the number itself**.

Simple Definition

A Perfect Number = Sum of its proper factors = Number itself

LOOK AT THIS!

Take **6**:

Factors of 6 (excluding 6 itself):

1, 2, 3

Now add them:

$$1 + 2 + 3 = 6 \quad \img alt="Green checkmark icon" data-bbox="223 608 246 631"/>$$

So, **6 is a Perfect Number!**

EASY ANALOGY

Think of 6 as a **pizza** 

Its pieces are:

$$\img alt="Pizza slice icon" data-bbox="118 783 141 806"/> 1 + \img alt="Pizza slice icon" data-bbox="178 783 201 806"/> 2 + \img alt="Pizza slice icon" data-bbox="238 783 261 806"/> 3 = \img alt="Whole pizza icon" data-bbox="298 783 321 806"/>$$

The pieces add up to the whole pizza!

REMEMBER!

Factors → Add → Compare

SLIDE 2 — FINDING THE FACTORS

HOW DO WE FIND FACTORS?

We check which numbers **divide the given number exactly**.

For example:

Number = 6

Number	6 % Number	Factor?
1	0	✓
2	0	✓
3	0	✓
4	2	✗
5	1	✗

So the proper factors are:

1, 2, 3

KEY IDEA

We can use:

None

```
n % i == 0
```

If the remainder is **0**, then **i** is a factor.

★ REMEMBER!

% → CHECKS DIVISIBILITY



SLIDE 3 — ADDING THE FACTORS

Once we find a factor, we add it to `sum`.



Example: 6

Start with:

None

```
sum = 0;
```

Check the numbers:

None

```
i = 1 → factor → sum = 1
```

```
i = 2 → factor → sum = 3
```

```
i = 3 → factor → sum = 6
```

So:

sum = 6

Then we compare:

None

```
if (sum == n)
```

If they are equal → **Perfect Number** ✓



BIG MEMORY TRICK

CHECK → ADD → COMPARE

- `%` → CHECK factor
- `sum = sum + i` → ADD factor
- `sum == n` → COMPARE



TAKEAWAY

Only add the **proper factors**.

So we usually check from:

None

```
i = 1
```

up to:

None

```
i < n
```

SLIDE 4 — CHECKING A PERFECT NUMBER

BASIC C CODE

None

```
int n, i, sum = 0;

scanf("%d", &n);

for (i = 1; i < n; i++)
{
    if (n % i == 0)
    {
        sum = sum + i;
    }
}

if (sum == n)
{
    printf("Perfect Number");
}
else
```

```
{  
    printf("Not Perfect Number");  
}
```

LET'S UNDERSTAND IT

For **n = 6**:

None

```
i = 1 → 6 % 1 = 0 → sum = 1  
i = 2 → 6 % 2 = 0 → sum = 3  
i = 3 → 6 % 3 = 0 → sum = 6  
i = 4 → not a factor  
i = 5 → not a factor
```

Finally:

None

```
sum = 6  
n   = 6
```

Therefore:

6 is a Perfect Number! 

★ MEMORY TRICK

LOOP → CHECK → ADD → COMPARE

SLIDE 5 — PERFECT NUMBER RECAP



 **REMEMBER THESE!**

Step	What we do
------	------------

- | | |
|---|--|
| 1 | Start <code>sum = 0</code> |
| 2 | Loop from <code>1</code> to <code>n-1</code> |
| 3 | Check <code>n % i == 0</code> |
| 4 | Add factor to <code>sum</code> |
| 5 | Compare <code>sum == n</code> |

BIG MEMORY TRICK

CHECK → ADD → COMPARE

None

`n % i == 0`

↓

FACTOR

↓

`sum = sum + i`

↓

`sum == n ?`

Question

1 Which of the following is a Perfect Number?

- A) 6
- B) 8
- C) 10
- D) 12



Hint: Find the proper factors and add them.

 Glossary:

Perfect Number → A number whose proper factors add up to the number itself.

2 What are the proper factors of 6?

- A) 1, 2, 3
- B) 1, 2, 3, 6
- C) 2, 3, 6
- D) 1, 6

 Hint: Do not include the number itself.

 Glossary:

Proper Factor → A factor of a number excluding the number itself.

3 Which statement adds a factor to the sum?

- A) `sum = i;`
- B) `sum = sum + i;`
- C) `sum = sum - i;`
- D) `sum = i / sum;`

 Hint: We need to keep the previous sum and add the new factor.

 Glossary:

Accumulate → Gradually build a total by adding values.

4 Why do we use $i < n$?

None

```
for (i = 1; i < n; i++)
```

- A) To include n as a proper factor
- B) To exclude n itself
- C) To stop at 1
- D) To skip all factors



Hint: A Perfect Number uses **proper factors**.



Glossary:

Exclude → Leave something out.

5 Find the Mistake

A student writes:

None

```
if (n % i == 1)
{
    sum = sum + i;
}
```

What is wrong?

- A) n should be sum
- B) i should be n

C) `== 1` should be `== 0`

D) Nothing is wrong



Hint: A factor gives a remainder of **zero**.



Glossary:

Remainder → The value left after division.